PROJETO: ARCOR QUIZ – “PENSE FORA DO CASCO.”

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA’S

Marco Antonio Iazzetti Cabral	26029566
Douglas Antônio de Almeida Caetano	26029717
Laura Sofia Hagiwara da Silva	26029361
Guilherme Rodrigues Nunes	26029691

São Paulo

2026

Sumário

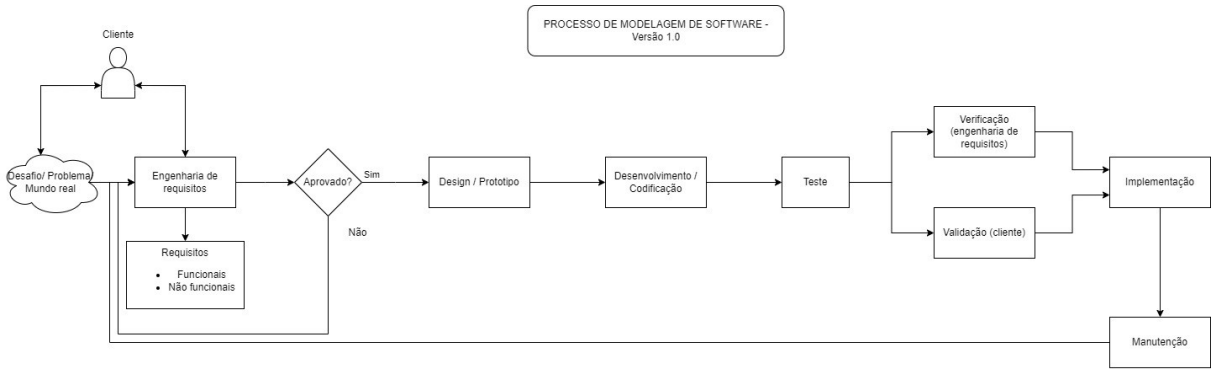
1 INTRODUÇÃO..... 3
1.1 – Processo de desenvolvimento de Software..... 3
1.2 Apresentação..... 3
2. DESIGN SPRINT – Ideação e prototipação do desafio..... 4
2.1 Desafio..... 4
2.2 Entender Mapear..... 4
2.2.1 – Entendimento e observação..... 4
2.2.2 – COMO PODEMOS?..... 4
2.2.3 – Pesquisa Desk..... 4
2.3 Ideação – desenho da solução (trilha do usuário)..... 5
2.3.1 “E se...?” → Geração de ideias doidas para inspirar ideias inovadoras..... 5
2.3.2 Crazy 8s (8 ideias em 8 minutos no papel)..... 5
2.3.3 Trilha do usuário..... 5
2.4 Prototipagem..... 7
3. REQUISITOS DE SISTEMA..... 8
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE..... 8
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE..... 10
4. DIAGRAMAS DE UML..... 13
4.1 CASOS DE USO..... 13
4.2 DIAGRAMA DE CLASSE..... 16
5. Especificação estrutura de uma rotina do sistema..... 18
5.1 – Apresentar a especificação de acordo com o modelo abaixo..... 18
5.2 – Apresentar a codificação da rotina..... 18
6. ARQUITETURA DO SISTEMA..... 20
6.1. Visão Geral do Sistema..... 20
6.2. Arquitetura Geral..... 20
6.3. Componentes Arquiteturais e Camadas..... 20
6.4. Diagrama da Arquitetura..... 22
6.5. Organização de Pastas e Módulos..... 23
6.6. Integrações Externas (se houver)..... 23
6.7. Ambiente e Implantação..... 23
7. REFERÊNCIAS BIBLIOGRÁFICAS..... 24

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

O processo de modelagem de software seguido pela equipe parte do desafio do mundo real, passa pela engenharia de requisitos (funcionais e não funcionais), pela aprovação do cliente, pelo design/protótipo, pelo desenvolvimento, pelos testes (verificação e validação) e termina na implementação e na manutenção.

Figura 1 – Processo de modelagem de software (versão 1.0)



Fonte: elaborado pela equipe no Draw.io (2026).

1.2 Apresentação

A Arcor (https://www.arcor.com.br/) atua no ramo alimentício com um portfólio que reúne chocolates, biscoitos, guloseimas e outras categorias. Como ação de marketing e relacionamento com o público, a empresa propõe às equipes do 1° semestre do curso de Análise e Desenvolvimento de Sistemas o desenvolvimento de um sistema digital com caráter lúdico e/ou educativo, capaz de aproximar o jogador da marca Arcor e de suas diferentes marcas, linhas e produtos.

A proposta central é transformar o conhecimento sobre o portfólio em uma experiência de jogo. O participante deverá avançar por perguntas, desafios, pistas e feedbacks que apresentem marcas como Tortuguita e Triunfo, além de outras marcas e produtos selecionados pela equipe em conjunto com o parceiro. O conteúdo utilizado deverá ser baseado em materiais oficiais da Arcor e validado antes da entrega final, evitando afirmações promocionais, nutricionais ou de qualidade que não estejam apoiadas em fontes fornecidas ou aprovadas pela empresa.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Proposta inicial do PI – Com o objetivo de apresentar a marca Arcor, referência no país no ramo de guloseimas, biscoitos e, principalmente, chocolates, foi proposto criar um jogo, em formato de quiz ou de descoberta.

Nele, seremos desafiados a lidar com banco de dados, linguagens de programação, lógica e, claro, um bom planejamento para uma execução assertiva.

2.2 Entender Mapear

2.2.1 – Entendimento e observação

- Entender a empresa e todos os seus segmentos;
- Entender seu público-alvo;
- Mapear como o usuário vai acessar o jogo; onde vai encontrá-lo.

2.2.2 – COMO PODEMOS?

- Como podemos compreender melhor os consumidores;
- Como podemos divulgar o jogo;
- Como podemos fazer um quiz que o usuário interaja de forma contínua;
- Como podemos criar um ranking online;
- Como podemos deixar o jogo chamativo;
- Como podemos diferenciar o jogo dos demais;
- Como podemos seguir inovando;
- Como podemos atrair os funcionários da empresa em geral;
- Como podemos criar critérios de avaliações;
- Como podemos entregar no tempo certo;
- Como podemos incluir o tempo como requisito de pontuação.

2.2.3 – Pesquisa Desk

- Sobre o quiz temos: IBGE – Educa | Crianças | Home; https://pt.quizur.com/trivia/marcas-de-chocolate-zkwm; https://www.educaplay.com/learning-resources/29028714-arcor_company_quiz.html;
- Sobre o layout e interações temos: Página inicial – arcor.com.br.

2.3 Ideação – desenho da solução (trilha do usuário)

GERAÇÃO DE IDEIAS.

2.3.1 "E se...?" → Geração de ideias doidas para inspirar ideias inovadoras

- Se nosso grupo tivesse super poderes, faríamos o jogo automaticamente;
- Se tivéssemos muito dinheiro, contrataríamos uma empresa para criar o jogo;
- Se não tivéssemos dinheiro nenhum, faríamos o quiz como estamos fazendo agora;
- E se fôssemos o Dick Vigarista e quiséssemos boicotar, iríamos boicotar os projetos dos outros grupos.

2.3.2 Crazy 8s (8 ideias em 8 minutos no papel)

- O ganhador vira CEO da Arcor;
- Os 3 primeiros têm um ano de chocolate grátis;
- O jogo tem que ser melhor que GTA 6;
- As perguntas serão acumulativas exponencialmente;
- Fase alternativa para o planeta chocolate;
- Só pode jogar comendo Tortuguita;
- Se responder em até 2 segundos, sua pontuação dobra;
- Mascote Tortuguita comemorando a cada acerto e incentivando o jogador a cada erro.

2.3.3 Trilha do usuário

Site do quiz sobre Arcor

Usuário: um jovem, apaixonado por chocolate, descobriu um site de quiz sobre a Arcor.

Etapa	Descobrir	Acessar	Escolher	Responder	Visualizar
O que o usuário faz	Abre o site	Faz login	Escolhe o nível do jogo	Responde questões	Visualiza sua pontuação
Como ele se sente	Curioso	Enfezado	Indeciso	Pensativo	Entusiasmado / frustrado
Onde ele trava	Link demora a abrir	Cadastro longo	Incerteza de escolher um nível difícil	Muito difícil	Não foi o esperado

Oportunidades que saem da última linha:

- **Link demora a abrir:** página inicial leve e carregamento rápido (RNF02);
- **Cadastro longo:** cadastro em uma única tela, com apenas 4 campos, e opção "Continuar com Google" (Telas 1 e 2);

- **Incerteza ao escolher o nível:** níveis com descrição curta e opção “Aleatório” (Tela 3);
- **Perguntas muito difíceis:** dica opcional em cada pergunta, com pequeno desconto de pontos (RFS05);
- **Resultado inesperado:** tela final com acertos, erros, tempo total e posição no ranking (Tela 5).

Figura 2 – Jornada do usuário: Quiz Arcor



Fonte: elaborado pela equipe (2026).

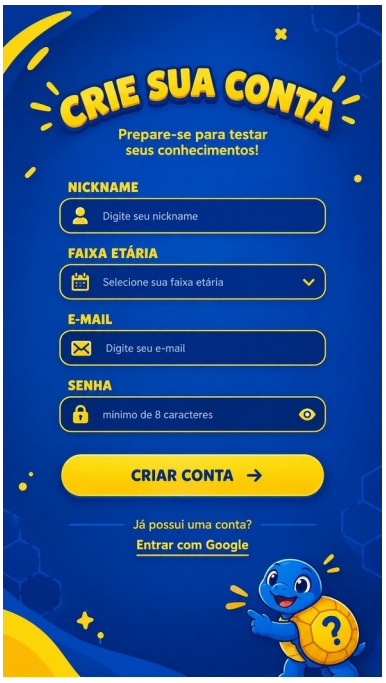
2.4 Prototipagem

Protótipo das telas principais do Arcor Quiz, em formato vertical, pensado para uso no celular. O fluxo segue a trilha do usuário: Bem-vindo → Criar conta (ou Google) → Escolha do nível → Perguntas → Resultados e pontuação.

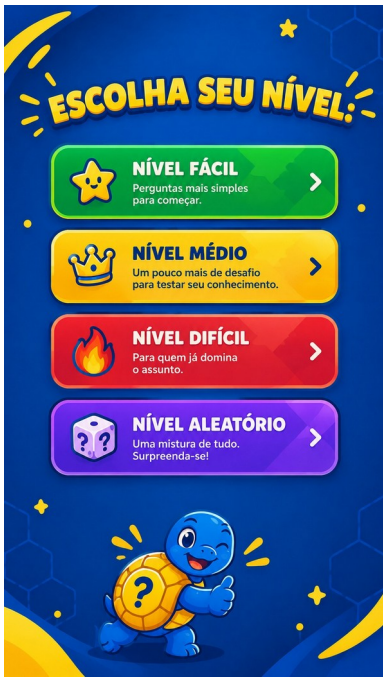
Tela 1 — Bem-vindo



Tela 2 — Criar conta



Tela 3 — Nível



Tela 4 — Perguntas



Tela 5 — Resultados



Legenda do fluxo

Tela 1: entrada com Google ou criação de conta.

Tela 2: nickname, faixa etária, e-mail e senha (mínimo de 8 caracteres).

Tela 3: níveis Fácil, Médio, Difícil e Aleatório.

Tela 4: pergunta com 4 alternativas (A–D), cronômetro e pontos.

Tela 5: pontuação, acertos, erros, tempo total e posição no ranking.

3. REQUISITOS DE SISTEMA

Os requisitos abaixo foram levantados a partir do desafio proposto pela Arcor, da trilha do usuário e do protótipo das telas. Cada requisito segue o modelo de especificação: função, descrição, entradas, fonte, saídas e ação.

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01 – Cadastrar jogador	
Função	Cadastrar jogador (Criar conta).
Descrição	Permite que um novo jogador crie sua conta de forma curta e rápida, para participar do quiz e aparecer no ranking.
Entradas	Nickname (texto), faixa etária (seleção), e-mail e senha (mínimo de 8 caracteres).
Fonte	Jogador, por meio do formulário “Criar conta” (Tela 2).
Saídas	Conta criada, sessão iniciada e abertura da escolha de nível (Tela 3); mensagem de erro indicando o campo inválido, quando houver.
Ação	1. Validar se todos os campos foram preenchidos, se o e-mail tem formato válido e se a senha tem pelo menos 8 caracteres. 2. Verificar se o e-mail e o nickname ainda não estão em uso. 3. Armazenar a senha criptografada (hash) e gravar o jogador. 4. Iniciar a sessão e abrir a Tela 3.

RFS02 – Autenticar jogador	
Função	Autenticar jogador (Entrar).
Descrição	Permite entrar no sistema com e-mail e senha ou com a conta Google, mantendo a sessão ativa durante o jogo.
Entradas	E-mail e senha, ou autorização da conta Google.
Fonte	Jogador (Tela 1 – “Continuar com Google” e formulário de login) e provedor Google (OAuth2).
Saídas	Sessão autenticada e acesso à Tela 3; mensagem “credenciais inválidas” em caso de falha.
Ação	1. Comparar a senha informada com o hash armazenado ou validar o token recebido do Google. 2. No primeiro acesso com Google, solicitar nickname e faixa etária e criar o jogador. 3. Gerar o token de sessão (JWT) com tempo de expiração. 4. Após 5 tentativas incorretas seguidas, bloquear novas tentativas por alguns minutos.

RFS03 – Selecionar nível de dificuldade	
Função	Selecionar nível e iniciar partida.
Descrição	Apresenta os níveis Fácil, Médio, Difícil e Aleatório, com uma descrição curta de cada um, para o jogador escolher o desafio.
Entradas	Nível escolhido (Fácil, Médio, Difícil ou Aleatório).
Fonte	Jogador, na Tela 3; banco de perguntas.
Saídas	Partida criada com o nível escolhido, 10 perguntas sorteadas e abertura da Tela 4.
Ação	1. Criar a partida (jogador, nível, data e hora de início). 2. Sortear 10 perguntas do banco, sem repetição; no nível Aleatório, misturar perguntas dos três níveis. 3. Iniciar o cronômetro da partida.

RFS04 – Responder perguntas	
Função	Apresentar perguntas e registrar respostas.
Descrição	Exibe uma pergunta por vez, com 4 alternativas (A, B, C e D), cronômetro e pontuação, e registra cada resposta do jogador.
Entradas	Alternativa escolhida (A a D) e tempo decorrido.
Fonte	Jogador (Tela 4) e banco de perguntas.
Saídas	Indicação de acerto ou erro, pontuação e contador (n/10) atualizados e próxima pergunta; após a 10ª, abertura da Tela 5.
Ação	1. Comparar a alternativa escolhida com a correta. 2. Registrar a resposta (certa ou errada, tempo gasto e se houve dica). 3. Somar 10 pontos a cada acerto. 4. Avançar para a próxima pergunta ou encerrar a partida.

RFS05 – Solicitar dica	
Função	Solicitar dica ao sistema.
Descrição	Permite que o jogador peça uma dica sobre a pergunta atual; cada dica usada desconta 3 pontos da pontuação final.
Entradas	Solicitação de dica (botão “Dica”), identificação da partida e da pergunta atual.
Fonte	Jogador (Tela 4) e banco de perguntas (texto da dica).
Saídas	Texto da dica, contador de dicas usadas e pontuação parcial atualizados.
Ação	1. Verificar se a partida está em andamento e se a pergunta ainda não foi respondida. 2. Se for a primeira dica da pergunta, somar 1 ao contador de dicas. 3. Se a dica já foi pedida nessa pergunta, reexibir sem novo desconto. 4. Detalhamento na Seção 5.

RFS06 – Calcular pontuação, exibir resultado e ranking	
Função	Finalizar partida e exibir resultado e ranking.
Descrição	Ao término das 10 perguntas, calcula a pontuação final (acertos × 10 − dicas × 3), mostra acertos, erros, tempo total e posição no ranking e oferece jogar novamente ou voltar ao menu.
Entradas	Respostas da partida, dicas usadas, tempo total (em segundos) e pontuações das demais partidas.
Fonte	Partida em andamento (Tela 4) e base do ranking.
Saídas	Tela 5 com pontuação, acertos e erros (quantidade e %), tempo total (mm:ss) e posição no ranking (exemplo: #12).
Ação	1. Finalizar a partida e calcular a pontuação (mínimo zero). 2. Gravar a partida e atualizar o ranking: maior pontuação primeiro e, em caso de empate, menor tempo total. 3. Calcular a posição do jogador e exibir a Tela 5. 4. “Jogar novamente” abre a Tela 3; “Voltar ao menu” retorna ao menu.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Nos requisitos não funcionais, “Entradas” indica a condição ou carga considerada, “Saídas” o critério de aceitação e “Ação” como o requisito será atendido e verificado.

RNF01 – Usabilidade e compatibilidade	
Função	Interface simples, intuitiva e responsiva.
Descrição	A interface deve ser colorida, com botões grandes e textos curtos, adequada ao público jovem, e se adaptar a celular, tablet e computador. O cadastro deve ter poucos campos.
Entradas	Toques e cliques do jogador em diferentes dispositivos e navegadores.
Fonte	Trilha do usuário (dores “cadastro longo” e “incerteza de escolher o nível”) e protótipo das telas.
Saídas	O jogador conclui o cadastro e a primeira partida sem ajuda; layout adaptado a telas de 360 px de largura ou mais; funcionamento nas versões atuais de Chrome, Edge, Firefox e Safari.
Ação	Desenvolver com abordagem mobile-first, testar em celulares e navegadores diferentes e validar com colegas (6° passo da trilha do usuário).

RNF02 – Desempenho	
Função	Carregar rápido e responder sem travar.
Descrição	O sistema deve abrir depressa e responder a cada ação do jogador sem atrasos perceptíveis.
Entradas	Até 200 jogadores simultâneos, em conexão móvel 4G.
Fonte	Trilha do usuário (dor “link demora a abrir”).
Saídas	Tela inicial carregada em até 3 segundos; 95% das requisições da API respondidas em até 1 segundo.
Ação	Usar páginas e imagens leves, consultas ao banco com índices e testes de carga antes da entrega final.

RNF03 – Segurança e privacidade	
Função	Proteger as contas e os dados dos jogadores.
Descrição	Senhas nunca são guardadas em texto puro; toda a comunicação é criptografada; são coletados apenas os dados necessários, de acordo com a LGPD (Lei nº 13.709/2018).
Entradas	Cadastro, login e requisições à API.
Fonte	Legislação (LGPD) e boas práticas de segurança de aplicações web.
Saídas	Senhas armazenadas com hash; tráfego via HTTPS; sessão por token com expiração; apenas o nickname aparece no ranking (e-mail nunca é exibido).
Ação	Aplicar hash de senha, HTTPS/TLS, consultas parametrizadas (contra injeção de SQL), faixa etária em vez de data de nascimento e consentimento do responsável para menores, quando aplicável.

RNF04 – Disponibilidade e confiabilidade	
Função	Manter o sistema disponível durante a campanha.
Descrição	O quiz deve ficar no ar durante todo o período de uso e não pode perder respostas e pontuações já registradas.
Entradas	Falhas de rede, reinício de serviços e picos de acesso.
Fonte	Necessidade da Arcor de manter a ação de marketing ativa e o ranking confiável.
Saídas	Disponibilidade mínima de 99% no período da campanha; backup diário do banco de dados; mensagens de erro claras, sem travar a tela.
Ação	Gravar resposta e pontuação em transações, agendar backups diários e monitorar o serviço.

RNF05 – Manutenibilidade e escalabilidade	
Função	Facilitar a manutenção e o crescimento do sistema.
Descrição	O código deve ser organizado em camadas e módulos, para permitir incluir novas perguntas, níveis ou marcas sem alterar a lógica principal.
Entradas	Novas funcionalidades, correções e aumento do número de jogadores.
Fonte	Proposta de seguir inovando e arquitetura definida na Seção 6.
Saídas	Perguntas cadastradas no banco (sem alterar código); regras de pontuação cobertas por testes automatizados; API sem estado, que permite replicação.
Ação	Versionar o código no Git, seguir padrão de nomes, separar as camadas e criar testes automatizados das regras de pontuação e dicas.

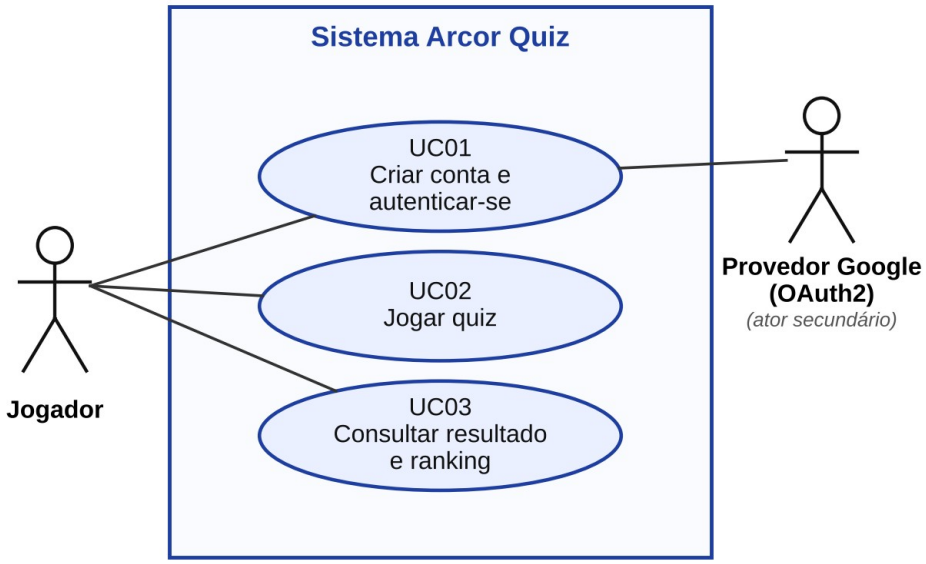
RNF06 – Integridade e conformidade do conteúdo	
Função	Garantir que o conteúdo do quiz seja confiável.
Descrição	Perguntas, dicas e respostas devem se basear em materiais oficiais da Arcor e ser validadas antes da entrega final, sem afirmações promocionais, nutricionais ou de qualidade sem fonte aprovada.
Entradas	Materiais oficiais fornecidos ou aprovados pela Arcor.
Fonte	Proposta do PI (Seção 1.2).
Saídas	Todas as 30 perguntas (10 por nível) com fonte registrada e validadas antes da entrega final.
Ação	Registrar a fonte oficial de cada pergunta, revisar o banco de perguntas com o parceiro e remover o que não tiver fonte.

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

O sistema possui um ator principal, o **Jogador**, e um ator secundário, o **Provedor Google (OAuth2)**, usado na entrada com a conta Google. Foram detalhados três casos de uso.

Figura 3 – Diagrama de casos de uso do Arcor Quiz



Fonte: elaborado pela equipe (2026).

UC01 – Criar conta e autenticar-se	
Atores	Jogador (principal); Provedor Google (secundário).
Descrição	O jogador cria uma conta ou entra no sistema para poder jogar.
Pré-condições	O jogador abriu o Arcor Quiz e tem acesso à internet.
Fluxo principal	1. O sistema exibe a Tela 1 (Bem-vindo). 2. O jogador escolhe “Criar uma conta”. 3. O jogador informa nickname, faixa etária, e-mail e senha (Tela 2). 4. O sistema valida os dados e verifica se o e-mail e o nickname estão disponíveis. 5. O sistema cria a conta e inicia a sessão. 6. O sistema exibe a escolha de nível (Tela 3).
Fluxos alternativos	A1. Continuar com Google: o sistema redireciona ao Google, recebe a autorização e entra (pedindo nickname e faixa etária no primeiro acesso). A2. Dados inválidos: o sistema indica o campo com erro e mantém os demais dados. A3. E-mail já cadastrado: o sistema informa e sugere entrar na conta. A4. Credenciais incorretas no login: mensagem de erro; após 5 tentativas, bloqueio temporário.
Pós-condições	Jogador autenticado, com sessão ativa.
Requisitos relacionados	RFS01, RFS02, RNF01, RNF03.

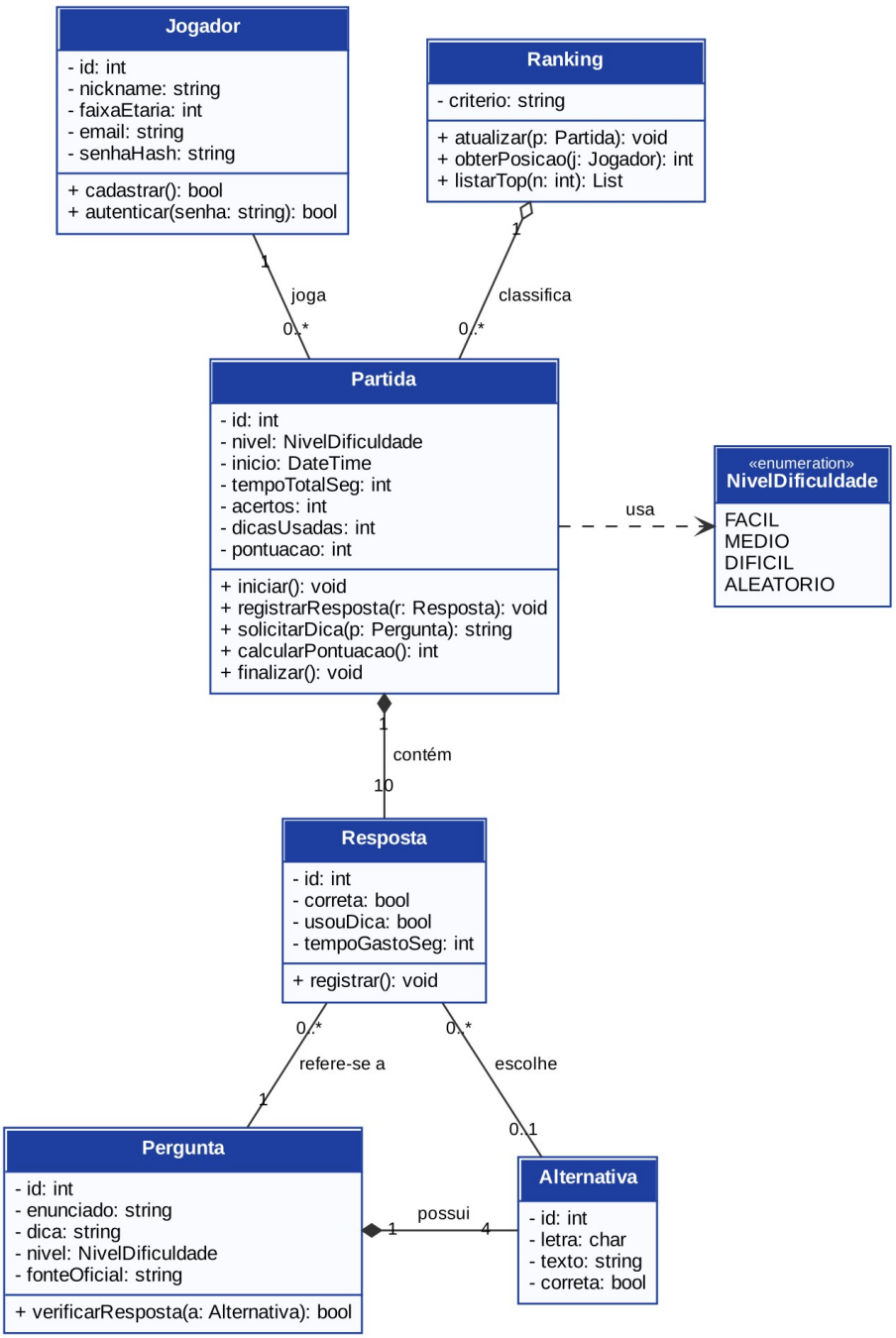
UC02 – Jogar quiz	
Atores	Jogador.
Descrição	O jogador escolhe um nível e responde a 10 perguntas, podendo pedir dicas.
Pré-condições	Jogador autenticado.
Fluxo principal	1. O sistema exibe os níveis Fácil, Médio, Difícil e Aleatório (Tela 3). 2. O jogador escolhe um nível. 3. O sistema cria a partida, sorteia 10 perguntas e inicia o cronômetro. 4. O sistema exibe a pergunta com 4 alternativas (Tela 4). 5. O jogador escolhe uma alternativa. 6. O sistema registra a resposta, indica acerto ou erro e atualiza os pontos. 7. Os passos 4 a 6 se repetem até a 10ª pergunta. 8. O sistema finaliza a partida e segue para o UC03.
Fluxos alternativos	A1. Solicitar dica (no passo 5): o sistema exibe a dica e desconta 3 pontos; pedir de novo na mesma pergunta não desconta outra vez. A2. Voltar ao menu (seta da Tela 4): a partida é abandonada e não entra no ranking. A3. Falha de conexão: o sistema mostra mensagem de erro e tenta reenviar a resposta; se não conseguir, cancela a partida.
Pós-condições	Partida concluída com 10 respostas registradas.
Requisitos relacionados	RFS03, RFS04, RFS05, RNF02, RNF06.

UC03 – Consultar resultado e ranking	
Atores	Jogador.
Descrição	Ao fim da partida, o jogador vê sua pontuação, o desempenho e a posição no ranking.
Pré-condições	Partida concluída (10 respostas registradas).
Fluxo principal	1. O sistema calcula a pontuação final (acertos × 10 – dicas × 3, mínimo zero). 2. O sistema grava a partida e atualiza o ranking. 3. O sistema exibe a Tela 5: pontuação, acertos, erros, tempo total e posição. 4. O jogador escolhe “Jogar novamente” (volta ao UC02) ou “Voltar ao menu”.
Fluxos alternativos	A1. Empate de pontuação: o sistema desempata pelo menor tempo total. A2. Falha ao atualizar o ranking: o sistema mostra o resultado sem a posição e tenta atualizar depois.
Pós-condições	Partida registrada e ranking atualizado.
Requisitos relacionados	RFS06, RNF04.

4.2 DIAGRAMA DE CLASSE

O diagrama de classes mostra as principais entidades do sistema, seus atributos, métodos e relacionamentos.

Figura 4 – Diagrama de classes do Arcor Quiz



Fonte: elaborado pela equipe (2026).

Principais classes:

- **Jogador:** pessoa cadastrada, com nickname, faixa etária, e-mail e senha (guardada como hash).
- **Partida:** uma rodada do quiz, com nível, tempo total, acertos, dicas usadas e pontuação; é quem calcula a pontuação e libera as dicas.
- **Resposta:** resposta dada a cada pergunta da partida (10 por partida), com tempo gasto e indicação de uso de dica.
- **Pergunta** e **Alternativa:** banco de perguntas; cada pergunta tem 4 alternativas, uma dica, um nível e a fonte oficial do conteúdo.
- **Ranking:** classifica as partidas por pontuação e, em empate, pelo menor tempo.
- **NivelDificuldade:** enumeração dos níveis; as perguntas são cadastradas como Fácil, Médio ou Difícil, e Aleatório é um modo de partida que mistura os três.

5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software: Arcor Quiz – módulo Jogar	
Função	Solicitar dica ao sistema.
Descrição	Entrega ao jogador a dica da pergunta atual e desconta 3 pontos da pontuação final. O desconto é aplicado uma única vez por pergunta.
Entradas	Partida (identificador, situação, acertos, dicas usadas e perguntas que já tiveram dica) e pergunta atual (identificador e dica).
Fonte	Botão “Dica” da Tela 4 (jogador) e banco de perguntas.
Saídas	Texto da dica, desconto aplicado (3 ou 0), total de dicas usadas e pontuação parcial.
Destino	Tela 4 (exibição ao jogador) e registro da partida no banco de dados.
Ação	• Verificar se a partida está em andamento. • Verificar se a pergunta atual ainda não foi respondida. • Verificar se a pergunta possui dica cadastrada. • Se for a primeira dica da pergunta: registrar a pergunta como “com dica”, somar 1 às dicas usadas e aplicar o desconto de 3 pontos. • Se a dica já foi usada nessa pergunta: apenas reexibir, sem novo desconto. • Recalcular a pontuação parcial (acertos × 10 − dicas × 3, mínimo zero) e devolver os dados.
Requisitos	RFS04, RFS05 e RFS06; RNF05 (regra coberta por teste automatizado).
Pré-condição	Jogador autenticado, partida em andamento, pergunta atual ainda não respondida e com dica cadastrada.
Pós-condições	Dica exibida; total de dicas usadas aumentado em 1 (somente na primeira solicitação da pergunta); pontuação parcial atualizada.
Efeitos colaterais	A pontuação final do jogador diminui 3 pontos por dica usada, o que pode alterar sua posição no ranking; a resposta da pergunta é marcada com “usou dica”.

5.2 – Apresentar a codificação da rotina

Pseudocódigo

```
FUNÇÃO solicitar_dica(partida, pergunta)
  SE partida NÃO está em andamento ENTÃO ERRO "A partida já foi finalizada"
  SE pergunta já foi respondida ENTÃO ERRO "A pergunta já foi respondida"
  SE pergunta NÃO possui dica ENTÃO ERRO "Esta pergunta não possui dica"
  desconto ← 0
  SE pergunta NÃO está em partida.perguntas_com_dica ENTÃO
    adicionar pergunta a partida.perguntas_com_dica
    partida.dicas_usadas ← partida.dicas_usadas + 1
    desconto ← 3
  FIM SE
  pontuacao_parcial ← MÁXIMO(partida.acertos × 10 − partida.dicas_usadas × 3, 0)
  RETORNAR pergunta.dica, desconto, partida.dicas_usadas, pontuacao_parcial
FIM FUNÇÃO
```

Python

```
from dataclasses import dataclass, field

PONTOS_POR_ACERTO = 10
DESCONTO_POR_DICA = 3

class ErroDica(Exception):
    """Erro de regra de negócio ao solicitar uma dica."""

@dataclass
class Pergunta:
    id: int
    enunciado: str
    dica: str

@dataclass
class Partida:
    id: int
    em_andamento: bool = True
    acertos: int = 0
    dicas_usadas: int = 0
    respondidas: set = field(default_factory=set)   # ids já respondidos
    com_dica: set = field(default_factory=set)      # ids que já usaram dica

def calcular_pontuacao(partida: Partida) -> int:
    """Pontuação = acertos * 10 - dicas usadas * 3 (nunca menor que zero)."""
    pontos = (partida.acertos * PONTOS_POR_ACERTO
              - partida.dicas_usadas * DESCONTO_POR_DICA)
    return max(pontos, 0)

def solicitar_dica(partida: Partida, pergunta: Pergunta) -> dict:
    """Entrega a dica da pergunta e desconta 3 pontos (1 vez por pergunta)."""
    if not partida.em_andamento:
        raise ErroDica("A partida já foi finalizada.")
    if pergunta.id in partida.respondidas:
        raise ErroDica("A pergunta já foi respondida.")
    if not pergunta.dica:
        raise ErroDica("Esta pergunta não possui dica cadastrada.")

    desconto = 0
    if pergunta.id not in partida.com_dica:      # primeira vez nesta pergunta
        partida.com_dica.add(pergunta.id)
        partida.dicas_usadas += 1
        desconto = DESCONTO_POR_DICA

    return {
        "dica": pergunta.dica,
        "desconto_aplicado": desconto,
        "dicas_usadas": partida.dicas_usadas,
        "pontuacao_parcial": calcular_pontuacao(partida),
    }

if __name__ == "__main__":
    partida = Partida(id=1, acertos=3)
    p4 = Pergunta(4, "Qual é o principal ingrediente do chocolate?",
                  "É uma semente de árvore tropical.")
    print(solicitar_dica(partida, p4))   # 1ª vez: desconta 3 pontos
    print(solicitar_dica(partida, p4))   # 2ª vez: reexibe, sem novo desconto
    partida.respondidas.add(4)
    try:
        solicitar_dica(partida, p4)
    except ErroDica as e:
        print("Erro:", e)
```

Exemplo de execução (jogador com 3 acertos pede a dica da pergunta 4, pede de novo e depois responde):

```
{'dica': 'É uma semente de árvore tropical.', 'desconto_aplicado': 3,
  'dicas_usadas': 1, 'pontuacao_parcial': 27}
{'dica': 'É uma semente de árvore tropical.', 'desconto_aplicado': 0,
  'dicas_usadas': 1, 'pontuacao_parcial': 27}
Erro: A pergunta já foi respondida.
```

6. ARQUITETURA DO SISTEMA

Esta seção descreve o projeto de arquitetura do Arcor Quiz, do front-end até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema:** Arcor Quiz – "Pense fora do casco."
- **Objetivo do Sistema:** aproximar o público das marcas Arcor (como Tortuguita e Triunfo) por meio de um quiz online com perguntas, dicas, pontuação e ranking.
- **Público-Alvo:** jovens e fãs de chocolate, consumidores das marcas Arcor e funcionários da empresa.
- **Principais Funcionalidades:** cadastro e login (inclusive com Google); escolha de nível (Fácil, Médio, Difícil e Aleatório); perguntas com 4 alternativas e dica opcional; pontuação com desconto por dica; tempo total da partida; ranking online.

6.2. Arquitetura Geral

Foi adotada a **Arquitetura em Camadas (Layered Architecture)**, em modelo cliente-servidor: uma aplicação web de página única (SPA) conversa com uma API REST por HTTPS/JSON. Há separação clara entre apresentação, aplicação (API), domínio (regras de negócio) e persistência. Cada camada só se comunica com a camada logo abaixo, e a camada de domínio não depende de nenhuma tecnologia de infraestrutura.

Essa escolha deixa o sistema modular (cada parte pode ser trocada ou testada sozinha), escalável (a API não guarda estado e pode ser replicada) e simples de manter por uma equipe de iniciantes.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias:** HTML5, CSS3, JavaScript e React (SPA, mobile-first).
- **Responsabilidades:**
  - Exibir as telas do protótipo: Bem-vindo, Criar conta, Nível, Perguntas e Resultados;
  - Validação inicial de dados (campos obrigatórios, formato do e-mail e senha com 8 ou mais caracteres);
  - Mostrar o cronômetro e a pontuação durante a partida;
  - Comunicação com a API via HTTPS/JSON, enviando o token de sessão em cada requisição.

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias:** C# com ASP.NET Core Web API (.NET 8).

- **Responsabilidades:**
  - Implementação dos casos de uso (serviços de aplicação) e orquestração entre as camadas;
  - Autenticação e autorização (JWT) e login com Google (OAuth2);
  - Comunicação com a camada de dados por meio de repositórios;
  - Exposição de endpoints RESTful, listados abaixo.

Método	Rota	Função	Requisito
POST	/api/jogadores	Cadastrar jogador	RFS01
POST	/api/auth/login	Entrar com e-mail e senha	RFS02
POST	/api/auth/google	Entrar com Google	RFS02
GET	/api/niveis	Listar níveis de dificuldade	RFS03
POST	/api/partidas	Iniciar partida e sortear 10 perguntas	RFS03
POST	/api/partidas/{id}/respostas	Registrar resposta	RFS04
POST	/api/partidas/{id}/dicas	Solicitar dica	RFS05
POST	/api/partidas/{id}/finalizar	Finalizar e calcular pontuação	RFS06
GET	/api/ranking	Consultar ranking e posição do jogador	RFS06

6.3.3. Camada de Domínio (Negócio)

- **Descrição:**
  - Contém as regras de negócio e os modelos de domínio;
  - Independente das demais camadas.
- **Componentes:**
  - Entidades: Jogador, Partida, Pergunta, Alternativa, Resposta e Ranking;
  - Casos de uso: CadastrarJogador, Autenticar, IniciarPartida, ResponderPergunta, SolicitarDica, FinalizarPartida e ConsultarRanking;
  - Regras de validação: 10 perguntas por partida; 10 pontos por acerto; 3 pontos de desconto por dica (uma vez por pergunta); pontuação mínima zero; desempate por menor tempo; senha com 8 ou mais caracteres.

6.3.4. Camada de Persistência (Banco de Dados)

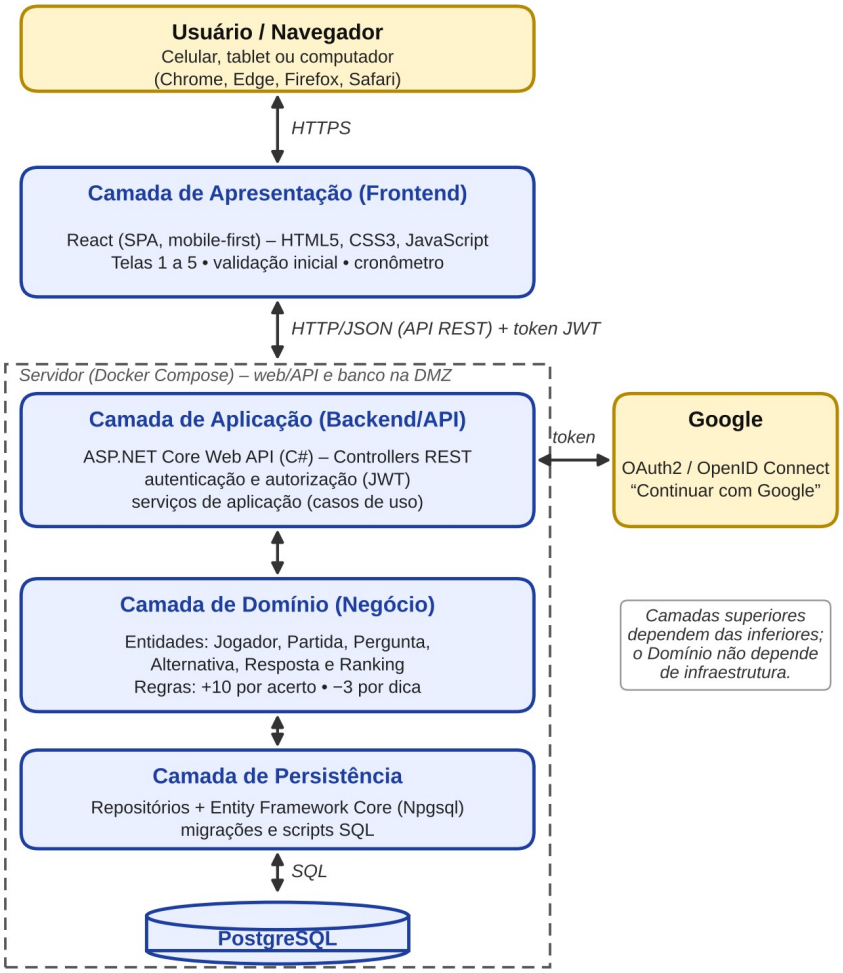
- **Tecnologias:** PostgreSQL, com Entity Framework Core (Npgsql).
- **Responsabilidades:**
  - Armazenamento e recuperação de dados nas tabelas jogador, partida, pergunta, alternativa e resposta (o ranking é obtido por consulta sobre as partidas);

- Scripts de criação e migração do banco e carga inicial (seed) do banco de perguntas: 10 perguntas por nível, cada uma com uma dica.

6.4. Diagrama da Arquitetura

O diagrama mostra o fluxo entre front-end, back-end e banco de dados, os componentes principais, a integração com o Google e a organização em contêineres.

Figura 5 – Diagrama da arquitetura do Arcor Quiz



Fonte: elaborado pela equipe (2026).

Exemplo textual da arquitetura:

```
[Usuário/Browser]
       |  HTTPS
  React (Frontend)
       |
   [HTTP/JSON + JWT]
       |
ASP.NET Core Web API (C#)  <---->  Google (OAuth2)
       |
  Domínio (regras) + Repositórios (EF Core)
       |  SQL
PostgreSQL (Banco de Dados)
```

6.5. Organização de Pastas e Módulos

Organização do código no repositório:

```
/frontend
├── src/
│   ├── components/   (Botao, CartaoPergunta, Cronometro, Mascote)
│   ├── pages/        (Bemvindo, CriarConta, EscolhaNivel, Perguntas, Resultado)
│   └── services/     (api, authService, partidaService)
/backend
├── ArcorQuiz.Api/            (Controllers/, Program.cs, appsettings.json)
├── ArcorQuiz.Application/    (Services/, DTOs/)
├── ArcorQuiz.Domain/         (Entities/, Rules/)
├── ArcorQuiz.Infrastructure/ (Data/, Repositories/, Migrations/)
└── ArcorQuiz.Tests/          (testes das regras de pontuação e dicas)
/database
├── schema.sql
├── seed_perguntas.sql
└── migrations/
docker-compose.yml
README.md
```

6.6. Integrações Externas (se houver)

- **Serviço de autenticação:** Google Identity (OAuth2 / OpenID Connect), usado no botão "Continuar com Google".
- Nesta versão não há integração com pagamentos, mapas, redes sociais ou serviços de envio de e-mail. O conteúdo das perguntas vem de materiais oficiais da Arcor e é carregado no banco por scripts, sem conexão direta com sistemas da empresa.

6.7. Ambiente e Implantação

- **Ambientes:** desenvolvimento (máquina da equipe), homologação (validação do conteúdo e do sistema com o professor e o parceiro) e produção.
- **Contêineres:** Docker Compose com três serviços: web (Nginx, que serve o front-end e encaminha as chamadas à API), api (ASP.NET Core) e db (PostgreSQL, com volume para os dados).
- **CI/CD:** GitHub Actions para compilar, rodar os testes e gerar as imagens a cada alteração no repositório.
- **Hospedagem:** nuvem compatível com .NET e Docker (por exemplo, Azure ou AWS); na apresentação acadêmica, execução local com Docker Compose.
- **Rede:** o servidor web e o servidor de banco de dados ficam na DMZ, atrás do firewall, conforme a topologia de rede definida na disciplina de Redes de Computadores.

7. REFERÊNCIAS BIBLIOGRÁFICAS

ARCOR. **Arcor Brasil**. Disponível em: https://www.arcor.com.br/. Acesso em: nov. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF, 2018.

EDUCAPLAY. **Arcor Company Quiz**. Disponível em: https://www.educaplay.com/learning-resources/29028714-arcor_company_quiz.html. Acesso em: nov. 2026.

FOWLER, M. **UML essencial**: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

IBGE. **IBGE Educa Crianças**. Disponível em: https://educa.ibge.gov.br/criancas. Acesso em: nov. 2026.

MARTIN, R. C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

MICROSOFT. **Documentação do ASP.NET Core**. Disponível em: https://learn.microsoft.com/aspnet/core/. Acesso em: nov. 2026.

QUIZUR. **Marcas de chocolate**. Disponível em: https://pt.quizur.com/trivia/marcas-de-chocolate-zkwm. Acesso em: nov. 2026.

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.